

CSIE 5374 Assignment 2 (Due on 05/09 23:59)

The page table of a task maps virtual memory addresses to physical memory addresses. In normal cases, page tables are managed by the OS kernel such as Linux, and are inaccessible from userspace. The Linux-specific system call `userfaultfd` (introduced in v4.3) allows a userspace program to receive events for page faults within registered memory regions. When a page fault occurs in such a region, the kernel sends a message on a file descriptor instead of handling the fault normally. Userspace can then resolve the fault.

Since `userfaultfd` delegates page-fault handling to userspace, we came up with a thinking: **Can we also perform the page table walk and leaf page table entry updates in userspace, instead of in the kernel?**

In assignment 2, we provide a `userfaultfd` program as a starting point. The purpose of this assignment is to explore how to expose and manipulate page tables in userspace by extending the Linux kernel and leveraging the `userfaultfd` mechanism. Inspired by [Dune](#), you will implement a new system call that remaps a target task's page tables into a userspace address range. Then, modify the `userfaultfd` program to walk and update those page tables, handling page faults entirely in userspace. (see Section `Userfaultfd Program` for more details)

Your modified Linux kernel and the test program must run on the same AArch64 machine emulated by QEMU, as you did in `env-setup`.

Development Tips

In the following sections, we assume you already know how to compile the Linux kernel, create a filesystem image, and run it with `qemu-system-aarch64`. Therefore, we will skip those steps already mentioned in `env-setup`.

Prerequisites

- Linux v6.1 source code
 - `git clone --depth 1 --branch v6.1 https://github.com/torvalds/linux.git`
- Cross compiler
 - `gcc-aarch64-linux-gnu (4:11.2.0-1ubuntu1)`
 - `sudo apt update && sudo apt install gcc-aarch64-linux-gnu`
- test program template
 - `userfaultfd.c`

NOTE: Ensure your kernel config sets `CONFIG_USERFAULTFD=y` and `CONFIG_TRANSPARENT_HUGEPAGE=n`.

Requirements

Add new system call to Linux (40%)

In this assignment, you are asked to add a new system call `remap_page_table` to Linux v6.1. The system call exposes a target process's page table page at a specified level to user space.

As discussed in the class, Arch64(Armv8) supports 48-bit addressing mode using four-level page tables. The different levels (from level 0, 1, 2, 3) of the page table are specified with the respective acronyms: PGD, PUD, PMD, PTE. You can check page 14 [here](#) and [the reference manual - VMSAv8](#) for how 4-level page table translation works in AArch64.

For the manual, look at **chapter D8.2 Translation process** to find out how Arm translates a virtual address to a physical address using four-level page tables. Figures D8-3 and D8-4 should include helpful information for you (You can ignore 52-bit support for now and focus on the 48-bit support, and check how Arm translates to a 4KB page). You can find Arm's page table descriptors (page table entries) in **chapter D8.3 Translation table descriptor formats**. As we discussed in the course, page table descriptors can be used to point to the next level of page table, or points to an entry. For the latter, the descriptor consists of the attribute bits (ex: for permission control) and the pfn bits (physical page frame of the translated physical address).

The specification of the system call is shown below. The system call takes one parameter, the `struct expose_pgtbl_args`. Please check the embedded comments below for what specific members in the

```
struct expose_pgtbl_args{
    // PID of the target task
    pid_t pid;
    // User-space virtual address to map the page table page
    unsigned long remap_vaddr;
    // Physical frame number from the previous level page table entry
    unsigned long pfn;
    // page table level (0: PGD, 1: PUD, 2: PMD, 3: PTE)
    unsigned int level;
}

SYSCALL_DEFINE1(remap_page_table, struct expose_pgtbl_args __user *, args)
{
    // Implement system call
}
```

Input validation: Return the error code `-EINVAL` if any of the following holds:

- Target PID does not exist.
- `pfn` is invalid or not aligned. (For level 0, it can be none.)
- `level` is out of range (0-3).
- `remap_vaddr` is not page-aligned.

Detailed Requirements:

- Kernel updates to the remapped page tables should be transparent to the userspace.

Hints:

- [remap_pfn_range](#)

Userfaultfd Program (45%)

In this section, you are asked to extend the provided `userfaultfd.c` template to build a complete page-fault handler in user space. The template uses `userfaultfd` to register a memory region and launches a `fault_handler` thread to catch page faults occurring within this region. Your goal is to complete the `va_to_pa` and `map_fault_va_to_pa` functions according to the **detailed requirements** below using the system call you added. After handling the page fault, issue the `UFFDIO_WAKE` ioctl to wake the main thread and resume execution.

Function: `va_to_pa` (20%)

```
unsigned long va_to_pa(unsigned long va)
```

This function performs a full page table walk in user space to translate a page-aligned virtual address (`va`) into its physical address (`pa`). The given `va` points to a page of size `PAGE_SIZE`.

Function: `map_fault_va_to_pa` (25%)

```
void map_fault_va_to_pa(unsigned long fault_va, unsigned long pa)
```

This function takes a faulting virtual address (`fault_va`) and a physical address (`pa`) obtained from `va_to_pa` . It performs a page table walk in user space by `fault_va` . Once you reach the leaf page table entry and detect it is invalid, overwrite it with a new entry pointing to `pa` . This update resolves the page fault without using other `userfaultfd` ioctls such as `UFFDIO_COPY` .

Example usage

```
unsigned long pa = va_to_pa(va);
map_fault_va_to_pa(fault_va, pa);
```

Detailed Requirements

- You should avoid duplicating the remap of the same page table.
- You can come up with your own way to allocate pages, and make use of the two functions to handle page faults.

NOTE: Do not use other `userfaultfd` ioctls such as `UFFDIO_COPY` for fault handling. All logic must rely on explicit page table walks and in-place entry updates in user space.

HINT: Refer to the Week 6 slide on page faults and check out when the `userfaultfd` passes the page fault to user space may be helpful.

Bonus (5%)

Simply replacing the leaf page table entry may lead to kernel bugs (e.g., `BUG: bad page map`) when the main thread unmaps the memory region. To get bonus points, you need to determine and implement additional steps in user space to prevent such issues.

WriteUp (10%)

Each of the groups is required to provide a write-up about the assignment. The write-up should include the instructions for how to compile and run your test programs, some explanation of your source code (e.g. description of how your code and test program work), and contributions from each of your group members.

Homework submission

You should submit the assignment via NTU Cool.

Submission Format

You are required to submit the kernel patch for Linux v6.1, the source code of your userspace program, and a write-up file. Compress all the files in `hw2.zip` , and upload the zip file to NTU Cool.

```
.
├─ write-up.md
├─ userfaultfd.c
├─ YOUR_6.1_KERNEL.patch
└─ any extra file
```

Each of the groups only needs to submit **one copy** of the assignment.

Grading criteria

Please run checkpatch against the changes you made to the Linux kernel. You will lose points if checkpatch reports either warnings or errors. Note that you are not asked to run checkpatch against your userspace programs.

In your implementation, you are required to properly protect shared memory accesses, manage resources (free allocated memory), handle errors, and return error codes.

We will also apply your submitted kernel patch to the Linux v6.1 kernel and test the patched kernel in our Ubuntu 22.04 environment. If we fail to apply your patch to the kernel source, or the patched Linux fails to compile, you will get **Zero** points.

Plagiarism policy

You are allowed to reference sources from the internet. If you do, please attach all of your references in the write-up. If we find that your code is similar to other's from the internet, we will count it as plagiarism if we cannot find the corresponding references. You will then get zero points for the assignment automatically.

Late Policy

We do not accept late submissions for this assignment. Please start the assignment early.